

Spark-Konfigurationseigenschaften auf Databricks setzen

Apache Spark lässt sich über zahlreiche sogenannte "Properties" (Konfigurationseigenschaften) feinjustieren. Diese Seite erklärt, wo und wie man solche Einstellungen auf Databricks vornimmt, welche davon bei Serverless Compute unterstützt werden und welche man besser gar nicht erst anfasst.

Grundregel: Möglichst wenig konfigurieren

Databricks empfiehlt ausdrücklich, die meisten Spark-Properties **nicht** manuell zu verändern. Der Grund: Viele dieser Einstellungen stammen noch aus älteren Spark-Versionen (Legacy-Einstellungen) und können die von Databricks bereits optimierten Standardwerte verschlechtern. Wer nicht genau weiß, was eine Einstellung bewirkt, sollte sie lieber unverändert lassen.

Auf welchen Ebenen kann man Konfigurationen setzen?

Es gibt zwei unterschiedliche Geltungsbereiche, je nachdem, wo eine Einstellung vorgenommen wird:

Ebene	Geltungsbereich
Compute-Ebene (Cluster)	Gilt für alle Notebooks und Jobs, die auf dieser Rechenressource laufen
Notebook-Ebene	Gilt nur für die aktuelle Spark-Session in diesem Notebook

Syntax je nach Sprache

Eine Konfigurationseigenschaft lässt sich in allen unterstützten Sprachen setzen. Beispiel anhand der Einstellung `spark.sql.ansi.enabled`:

- **SQL:** `SET spark.sql.ansi.enabled = true`
- **Python / Scala:** `spark.conf.set("spark.sql.ansi.enabled", "true")`

Um den aktuell gesetzten Wert einer Eigenschaft abzufragen, verwendet man:

```
spark.conf.get("configuration_name")
```

Unterstützte Properties bei Serverless Compute

Bei Serverless Compute ist die Menge an konfigurierbaren Properties bewusst eingeschränkt. Folgende sechs Eigenschaften lassen sich dort anpassen:

Property	Standardwert
<code>spark.databricks.execution.timeout</code>	9000 Sekunden (für Notebooks)
<code>spark.sql.ansi.enabled</code>	true
<code>spark.sql.shuffle.partitions</code>	auto
<code>spark.sql.session.timeZone</code>	Etc/UTC
<code>spark.sql.legacy.timeParserPolicy</code>	CORRECTED
<code>spark.sql.files.maxPartitionBytes</code>	128 MB

Nicht unterstützte Properties

Einige klassische Spark-Properties funktionieren auf Databricks nicht mehr, weil deren Aufgabe von der Plattform automatisch übernommen wird. Die Dokumentation nennt acht solcher Fälle, unter anderem:

- **Dynamic-Allocation-Properties** → Stattdessen übernimmt die automatische Databricks-Autoskalierung diese Aufgabe.
- `spark.master`, `spark.driver.*` → Werden automatisch von Databricks verwaltet, eine manuelle Einstellung ist nicht nötig bzw. nicht möglich.
- `spark.jars` → Stattdessen sollten JAR-basierte Tasks (Jobs) verwendet werden, um zusätzlichen Code einzubinden.

Spezialfälle: Databricks SQL und Lakeflow-Pipelines

- **Databricks SQL:** Bietet vereinfachte Aliase für die am häufigsten benötigten Konfigurationseigenschaften, sodass man nicht immer den vollständigen technischen Namen kennen muss.

- **Lakeflow-Pipelines:** Hier lassen sich Spark-Konfigurationen über die Benutzeroberfläche, per JSON-Definition oder über den `spark_conf` -Decorator im Code setzen.

Praktische Einordnung

Für Einsteiger ist meist am wichtigsten zu wissen: Es gibt einen Unterschied zwischen Cluster-weiten und Notebook-lokalen Einstellungen, die Syntax unterscheidet sich leicht je nach Sprache, und bei Serverless Compute ist die Auswahl an änderbaren Properties bewusst klein gehalten, um die Plattform stabil und vorhersehbar zu halten.

Quelle: <https://docs.databricks.com/aws/en/spark/conf>

Stand: 2026-08-06